Università degli Studi di Udine

Dipartimento di Scienze Matematiche,
Informatiche e Fisiche

IBML - Internet of Things, Big Data, Machine Learning

Internet of Things
Progetto Pet Tracker

Autori

Andrea Gioia, 169484	169484@spes.uniud.it
Alessio Nicodemo, 166972	166972@spes.uniud.it
Mattia Nadal, 167372	167372@spes.uniud.it
Francesco Sabot, 167339	167339@spes.uniud.it

Abstract

Il progetto Pet-Tracker nasce con l'obiettivo di realizzare un sistema di localizzazione GPS in tempo reale per animali domestici di media taglia, in grado di trasmettere la posizione, lo stato della batteria ed il numero di passi verso un server remoto accessibile tramite internet, e di visualizzare tali dati attraverso un'applicazione dedicata.

Indice

1 Architettura del sistema **2**
1.1 Mappatura sul modello IoTWF 2

2 Hardware **2**
2.1 LiLyGo T-SIM7670G-S3 2
2.2 Antenna GPS e LTE 2
2.3 Batteria e Power Management 2
2.4 BMA400 CJMCU-400 3

3 Firmware: Logica di funzionamento **3**

4 Back-end e Database **3**
4.1 Architettura di Hosting e Network Exposure 3
4.2 Sistema di Backup e Data Retention 3
4.3 Monitoraggio e Business Continuity 4
4.4 Struttura dei dati 4
4.5 Interfaccia API e API Rules 4
4.5.1 API Rules e Sicurezza 4
4.6 Smistamento e Normalizzazione 4
4.6.1 Funzionamento del Trigger onRecordAfterCreateSuccess 4
4.6.2 Vantaggi Architetturali e Corrispondenza IoTWF 5

5 Front-end e Applicazione Mobile **5**

6 Test e Risultati **5**

7 Conclusioni **5**

Riferimenti bibliografici **6**

1 Architettura del sistema

L'architettura del sistema Pet-Tracker segue il **Modello a 7 livelli dell'IoT World Forum (IoTWF)**, garantendo una transizione fluida dal dato fisico all'informazione per l'utente.

1.1 Mappatura sul modello IoTWF

- **Livello 1 (Physical Devices):** Rappresentato dalla board LilyGo e dai sensori (GPS, Accelerometro).
- **Livello 2 (Connectivity):** Gestito tramite il modem LTE e protocolli di trasporto sicuri.
- **Livello 3 (Edge Computing):** Elaborazione locale sul microcontrollore per il filtraggio delle stringhe NMEA e il calcolo dei passi.
- **Livello 4 (Data Accumulation):** Persistenza dei dati sul server PocketBase (database SQLite).
- **Livello 5 (Data Abstraction):** Esposizione dei dati tramite API REST strutturate in formato JSON.
- **Livello 6 (Application):** L'applicazione mobile dedicata che trasforma le coordinate in posizioni su mappa.
- **Livello 7 (Collaboration and Processes):** L'interazione del proprietario con il sistema per il monitoraggio dell'animale.

2 Hardware

2.1 LiLyGo T-SIM7670G-S3

Il dispositivo utilizza una board LiLyGo T-SIM7670G-S3 [1] che si basa sul modulo **ESP32-S3** dual-core, che gestisce sia la logica applicativa che la comunicazione con il modem SIM7670G tramite interfaccia UART.

2.2 Antenna GPS e LTE

Il sistema utilizza un'antenna flessibile LTE [2] per la connettività dati e un'antenna ceramica attiva [3] per il ricevitore GNSS integrato, essenziale per la precisione della localizzazione outdoor.

2.3 Batteria e Power Management

Viene impiegata una batteria ricaricabile 3000mAh 30A Litio-Ion ad alta capacità. Il monitoraggio avviene tramite un partitore di tensione collegato al pin `ADC_BAT` (GPIO 4), permettendo di calcolare la percentuale di carica residua.

2.4 BMA400 CJMCU-400

L'accelerometro usato è un **BMA400** [4] integrato nella breadboard **CJMCU-400**, che viene interrogato per rilevare il movimento, consentendo l'implementazione di un contapassi utile al monitoraggio della salute dell'animale.

3 Firmware: Logica di funzionamento

Il firmware, sviluppato in ambiente Arduino, implementa una macchina a stati per l'acquisizione e l'invio:

1. **Inizializzazione:** Configurazione dei pin, avvio della seriale hardware per il modem e attivazione del motore GNSS tramite comandi AT [5] (`AT+CGNSSPWR=1`).
2. **Loop Principale:**
   - Lettura dei dati GNSS tramite il comando `AT+CGNSSINFO`.
   - Acquisizione dello stato batteria (`leggiBatteria()`).
   - Parsing dei dati per verificare la validità del fix GPS.
3. **Trasmissione:** Se il fix è valido, i dati vengono incapsulati in una stringa JSON e inviati tramite una richiesta `HTTPS POST` all'endpoint di PocketBase.

4 Back-end e Database

Il centro nevralgico della gestione dati è basato su **PocketBase** [6], una soluzione open-source che integra un database embedded SQLite con un'interfaccia API REST.

4.1 Architettura di Hosting e Network Exposure

L'istanza di PocketBase è stata ospitata su una workstation locale e data la natura del sistema IoT, che richiede la raggiungibilità del backend da parte del dispositivo (tramite rete LTE) e dell'applicazione mobile (tramite rete dati/Wi-Fi), si è reso necessario esporre il servizio locale su rete pubblica. Il problema del *NAT Traversal* e dell'assenza di un indirizzo IP pubblico statico sulla rete locale è stato risolto mediante l'impiego di **ngrok** [7]. Questa tecnologia di *tunneling* crea un tunnel sicuro (HTTPS) tra l'endpoint pubblico di ngrok e la porta locale su cui è in esecuzione il server. Tale URL funge da *Reverse Proxy*, reindirizzando il traffico HTTPS in ingresso verso l'istanza locale di PocketBase, garantendo così la persistenza dei dati e l'accessibilità dei servizi in tempo reale ovunque sia presente una connessione internet.

4.2 Sistema di Backup e Data Retention

Per garantire la resilienza dei dati e prevenire perdite accidentali dovute a guasti del server locale, è stato implementato un sistema di backup automatizzato. Il processo utilizza l'utility **rclone** [8], uno strumento a riga di comando per la gestione del cloud storage, configurato per eseguire la sincronizzazione della directory dei dati di PocketBase verso un bucket remoto su **Backblaze B2** [9]. L'automazione è gestita tramite un *cron job* che esegue il backup in modo incrementale ogni primo giorno del mese.

4.3 Monitoraggio e Business Continuity

Per garantire l'affidabilità dell'intero ecosistema, è stato implementato un sistema di monitoraggio esterno tramite **UptimeRobot** [10]. Dato che l'accessibilità del backend dipende dalla stabilità del tunnel ngrok, il monitoraggio interroga l'endpoint HTTPS a intervalli regolari, inviando notifiche immediate in caso di downtime.

4.4 Struttura dei dati

I dati ricevuti vengono normalizzati e memorizzati in una collezione denominata `positions`. La struttura del record prevede i seguenti campi:

- `timestamp`: riferimento temporale dell'acquisizione.
- `lat` / `lon`: coordinate geografiche acquisite dal modulo GNSS;
- `battery`: stato di carica della batteria (espresso in voltaggio);
- `battery_percent`: stato di carica della batteria (espresso in percentuale);
- `charging`: booleano che indica se la batteria è in corso di ricarica;
- `feet`: conteggio dei passi rilevato dall'accelerometro BMA400;

4.5 Interfaccia API e API Rules

L'accesso ai dati memorizzati nel backend avviene esclusivamente tramite un'interfaccia **RESTful API** fornita nativamente da PocketBase. Questo approccio permette di separare nettamente il livello di persistenza (Database) dal livello di fruizione (Applicazione Mobile).

4.5.1 API Rules e Sicurezza

Per garantire l'integrità e la riservatezza dei dati, sono state configurate specifiche **API Rules** basate su espressioni booleane:

- **List/View:** Visualizzazione permessa solo agli utenti autenticati.
- **Create:** Abilitata per il dispositivo LilyGo tramite autenticazione dedicata.
- **Update/Delete:** Ristrette ai soli amministratori del sistema.

4.6 Logica di Smistamento e Normalizzazione (Server-side Hooks)

Per ottimizzare la gestione dei dati e garantire la separazione delle responsabilità, è stata implementata una logica di smistamento automatico lato server tramite i *JavaScript Event Hooks* di PocketBase. Questa architettura permette di mantenere la collezione `data_sent_raw` come un punto di accumulo "grezzo", delegando al server il compito di distribuire le informazioni nelle tabelle specializzate.

4.6.1 Funzionamento del Trigger onRecordAfterCreateSuccess

Il sistema sfrutta l'hook `onRecordAfterCreateSuccess`. Il processo di smistamento opera come segue:

- **Monitoraggio Energetico**: I parametri di tensione e carica vengono isolati nella collezione `battery_data`.
- **Tracciamento Geografico**: Le coordinate vengono duplicate in `positions_duplicate`, mantenendo un riferimento univoco (`raw_data_id`).
- **Analisi dell'Attività**: I dati del contapassi vengono indirizzati alla collezione `activities`.

4.6.2 Vantaggi Architetturali e Corrispondenza IoTWF

- **Efficienza del Livello 3 (Edge)**: Riducendo il numero di richieste HTTPS (invio singolo "bulk"), si preserva la batteria della LilyGo.
- **Ottimizzazione del Livello 5 (Abstraction)**: L'app Flutter interroga API già strutturate, garantendo fluidità sulla mappa.
- **Resilienza**: L'uso di `try-catch` assicura che errori su sensori secondari non compromettano il salvataggio del GPS.

5 Front-end e Applicazione Mobile

L'applicazione mobile, sviluppata in Flutter [11], interagisce con i dati tramite il PocketBase SDK, garantendo una comunicazione asincrona e performante.

6 Test e Risultati

7 Conclusioni

Riferimenti bibliografici

[1] *T-SIM7670G-S3 Specifications*. URL: https://github.com/Xinyuan-LilyGO/LilyGo-Modem-Series/blob/main/docs/en/esp32s3/sim7670g-s3/REAMDE.MD.

[2] *LTE Antenna Specifications*. URL: https://github.com/Xinyuan-LilyGO/LilyGo-Modem-Series/blob/main/datasheet/LTE%20Antenna%20Specifications.pdf.

[3] *GPS Antenna Specifications*. URL: https://github.com/Xinyuan-LilyGO/LilyGo-Modem-Series/blob/main/datasheet/GPS%20Antenna%20Specifications.pdfp.

[4] *BMA400 CJMCU-400 Datasheet*. URL: https://www.bosch-sensortec.com/media/boschsensortec/downloads/product_flyer/bst-bma400-fl000.pdf.

[5] *SIM7670 Series AT Command Manual*. URL: https://files.waveshare.com/wiki/SIM7670G-LTE-Cat-1-GNSS-HAT/SIM767XX_Series_AT_Command_Manual_V1.01.pdf.

[6] *PocketBase Documentation*. URL: https://pocketbase.io/docs/.

[7] *ngrok Documentation - HTTP Tunnels*. URL: https://ngrok.com/docs/secure-tunnels/.

[8] *rclone - rsync for cloud storage*. URL: https://rclone.org/.

[9] *Backblaze B2 Cloud Storage*. URL: https://www.backblaze.com/b2/cloud-storage.html.

[10] *UptimeRobot - Free Website Monitoring Service*. URL: https://uptimerobot.com/.

[11] *Flutter Documentation*. URL: https://docs.flutter.dev/.